Programmation 3D
Shaders

Objectif

L'objectif de ce cours est de comprendre la structure d'un shader et d'apprendre les bases de la programmation de shaders.

Définition

Définition

Le shader est un script exécuté par le GPU.

Il définit les règles utilisées pour rendre un mesh avec :

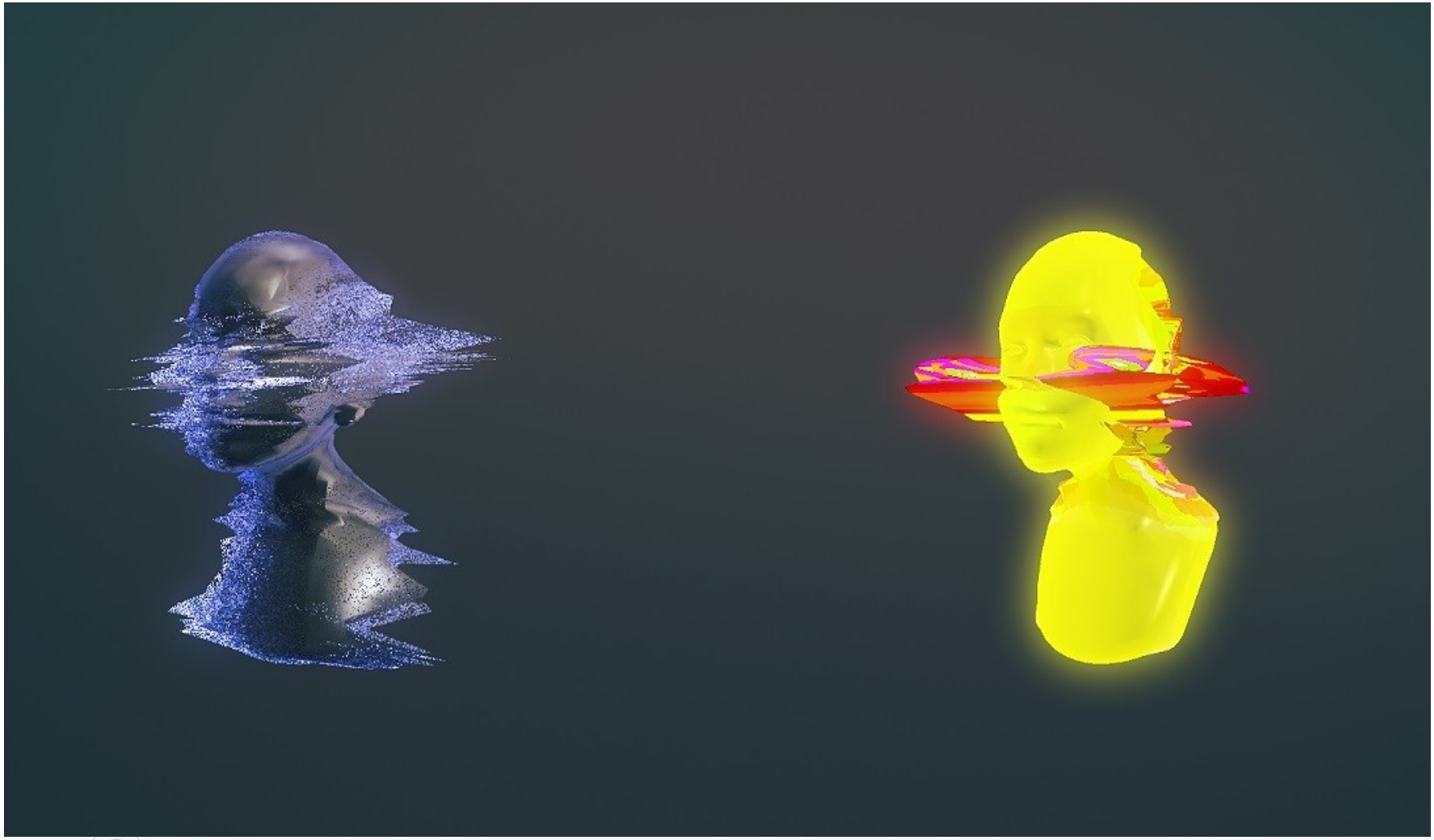
- Une configuration (material)
- Les données de l'environnement (caméra, lumières, ...).

Des possibilités infinies

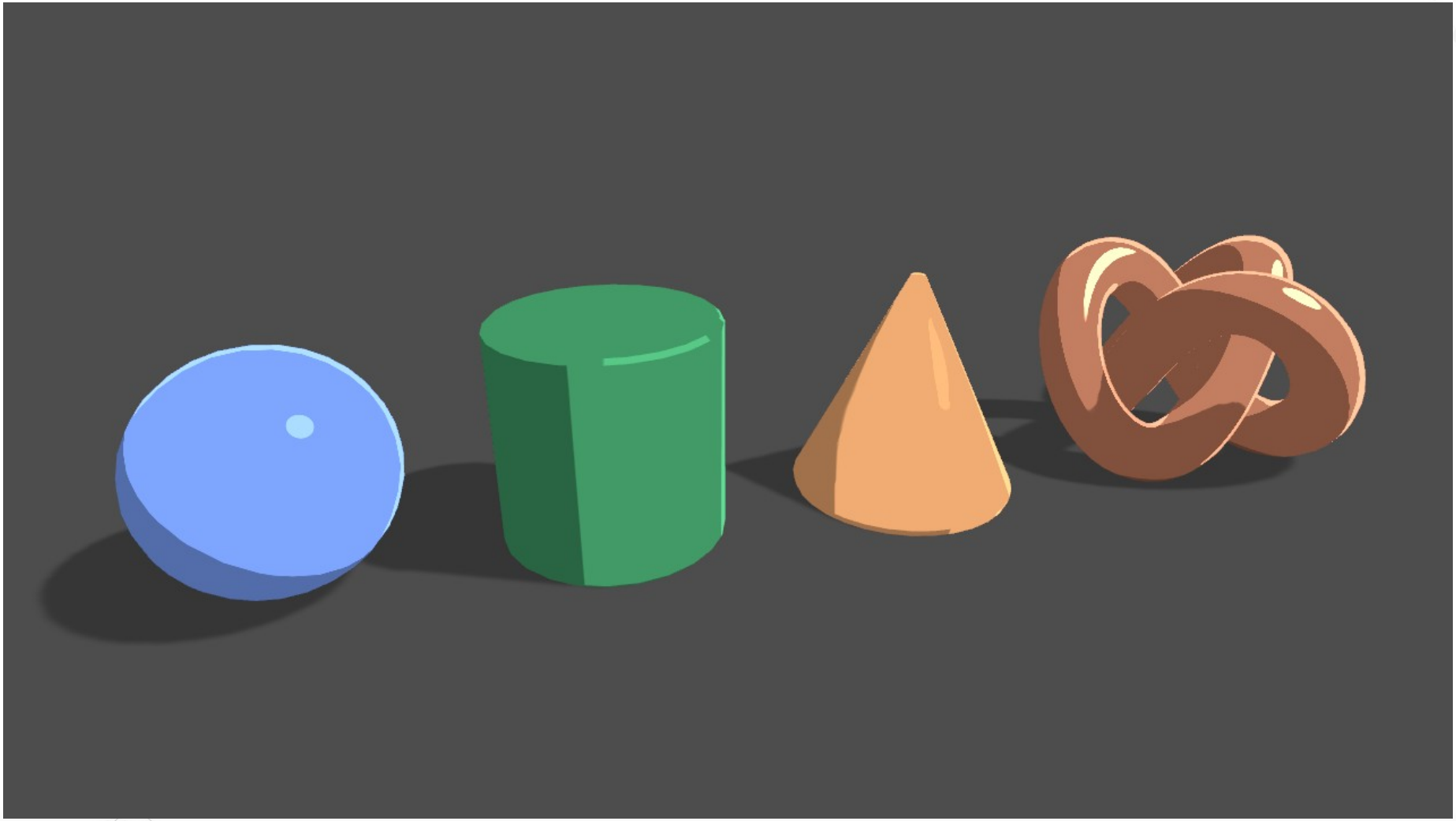
Exemples

Exemples

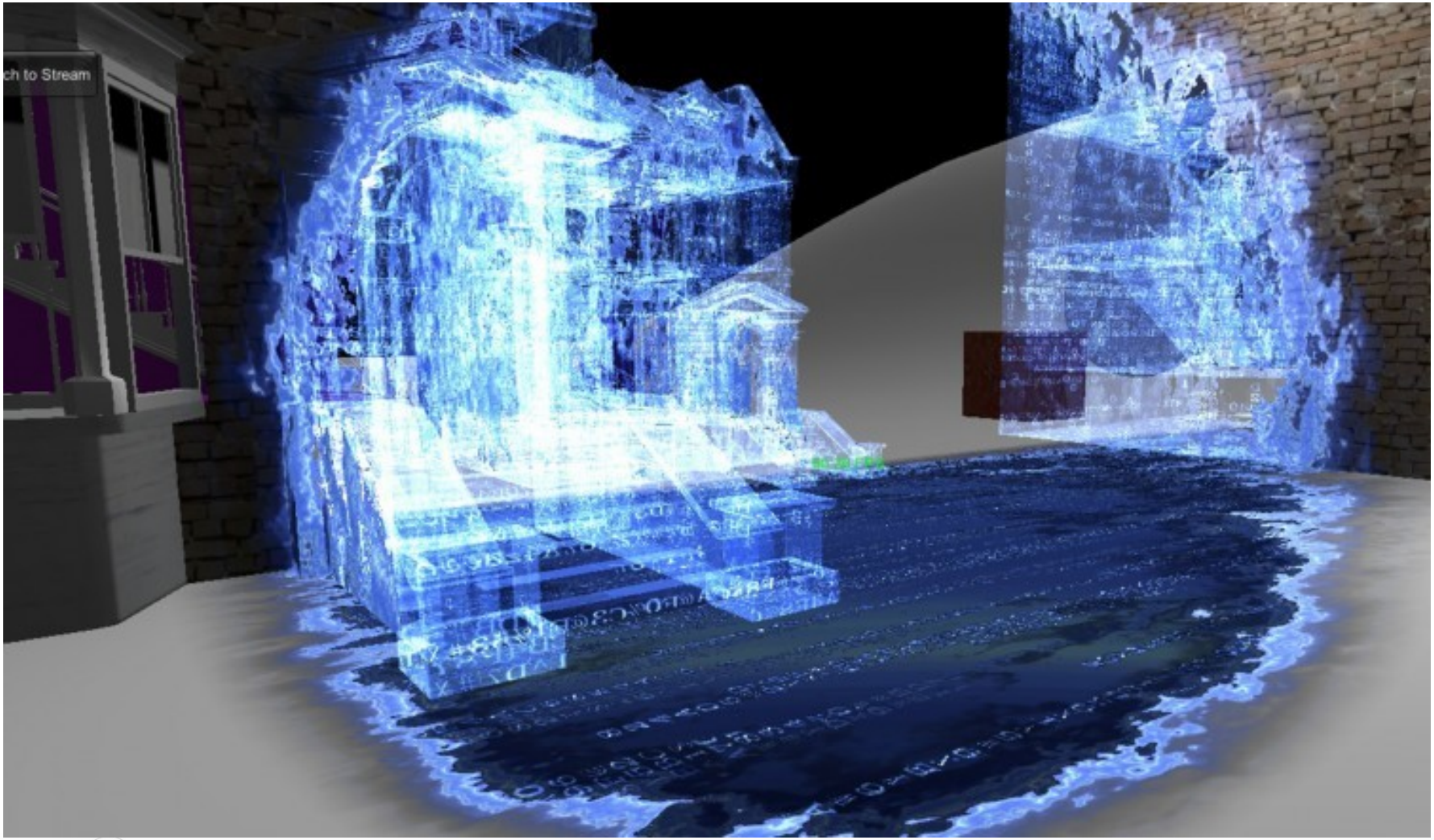
Exemples



Exemples



Exemples



Etapes d'exécution

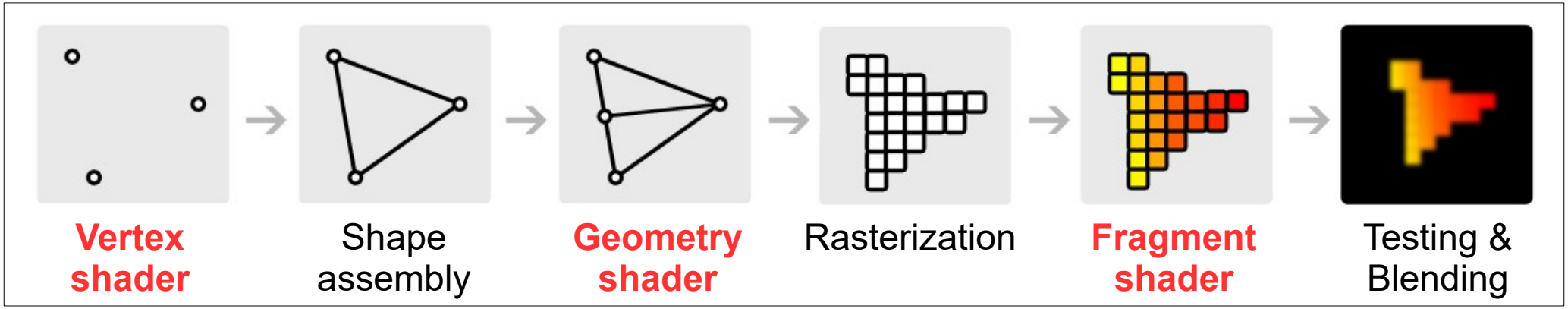
Exécution d'un shader

Lors du rendu d'un objet 3D, le shader parcours et modifie :

- Chaque vertex du mesh
- Chaque triangle du mesh
- Chaque pixel occupé par le mesh sur la surface de rendu.

Etapes

Mesh Data
+
Parameters



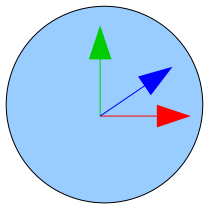
Projections

Le shader utilise :

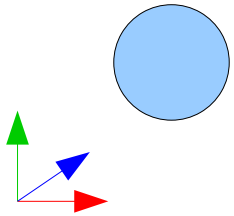
- Les informations de la caméra (position, rotation, fov, ...)
- La transformation de l'objet 3D (position, rotation, scale)

Pour projeter l'objet 3D rendu de son repère local au repère "Screen Space".

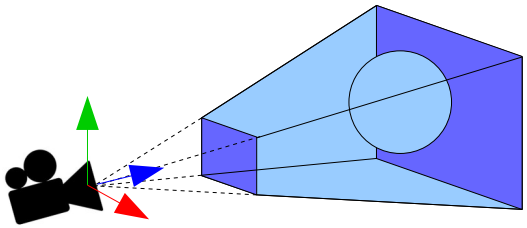
Projections



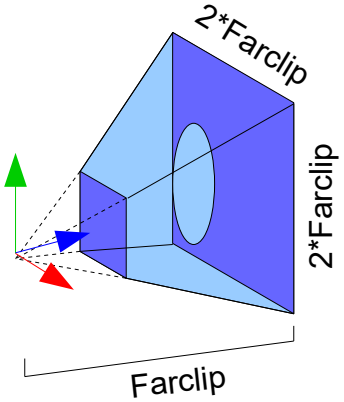
1 - Object Space



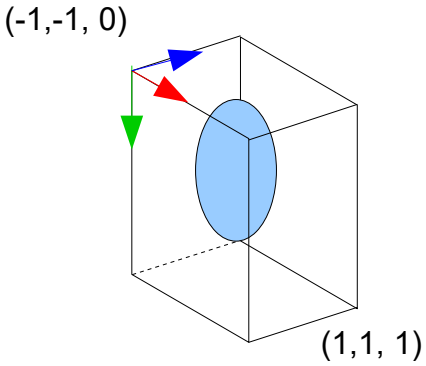
2 - World Space



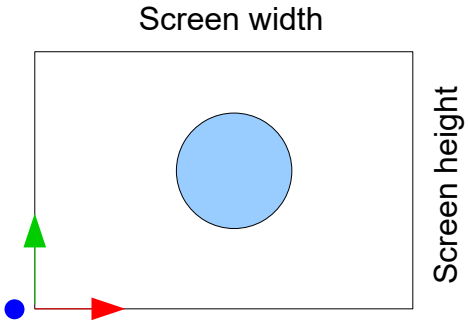
3 - Camera Space



4 - Clip Space (D3D)



5 - NDC Space (D3D)



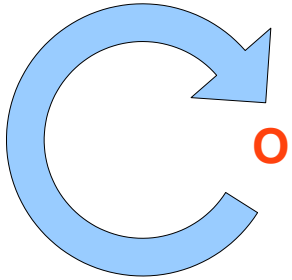
6 - Screen Space

Vertex shader

La fonction de **vertex shader** traite **chaque vertex** du mesh **indépendamment.**

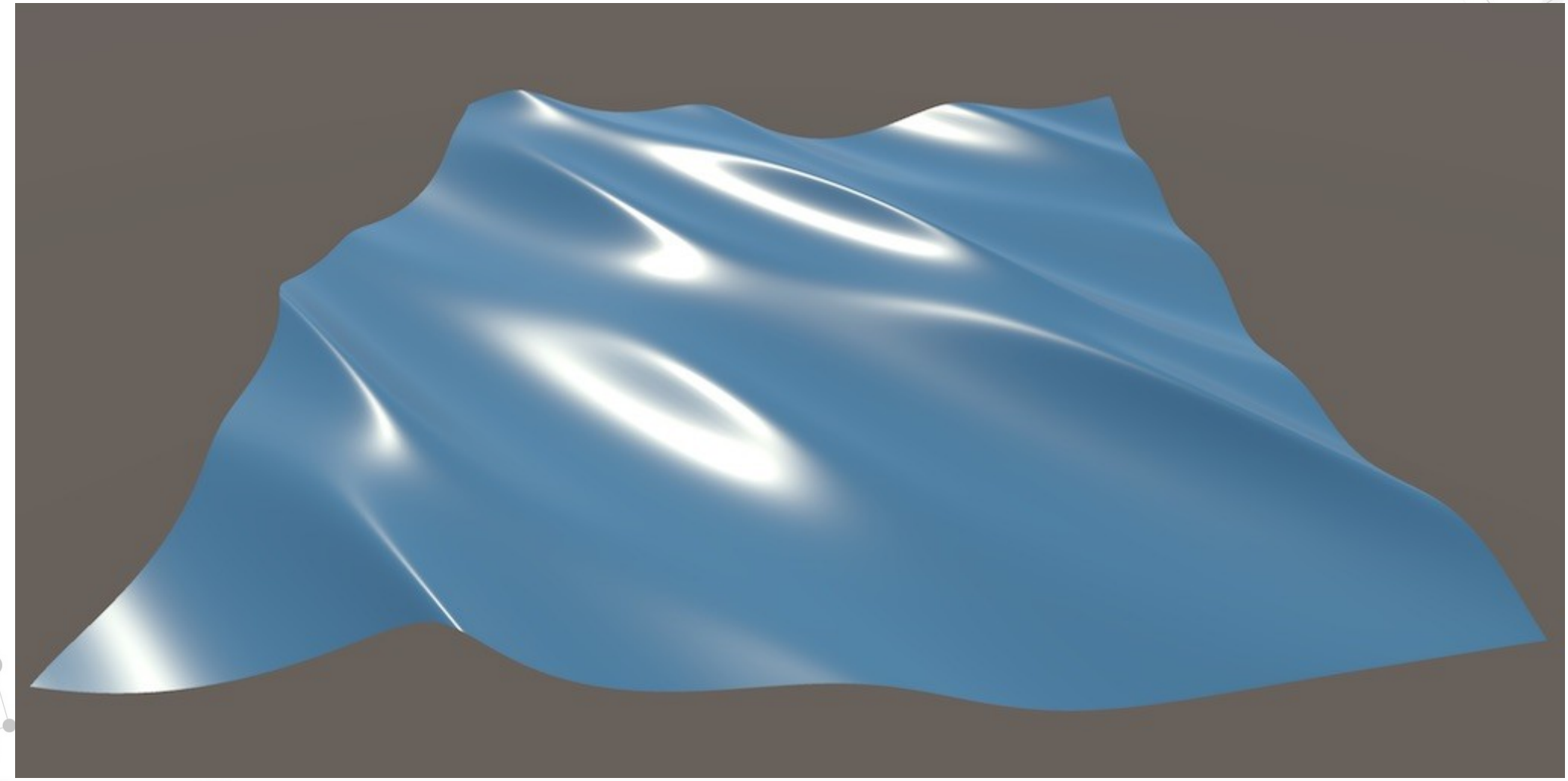
Elle permet de modifier et compléter les données contenues par chaque vertex (position, normale, couleur, ...).

ForEach (VertexData v in vertices)



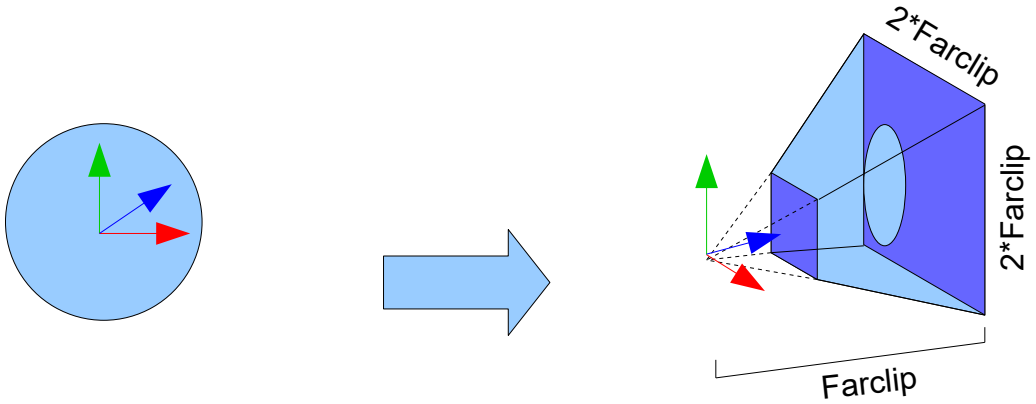
OutData o = vert(v)

Vertex shader



Vertex shader : Projections

Le vertex shader reçoit des positions de vertex en **Object Space** et renvoie des positions en **Clip Space.**



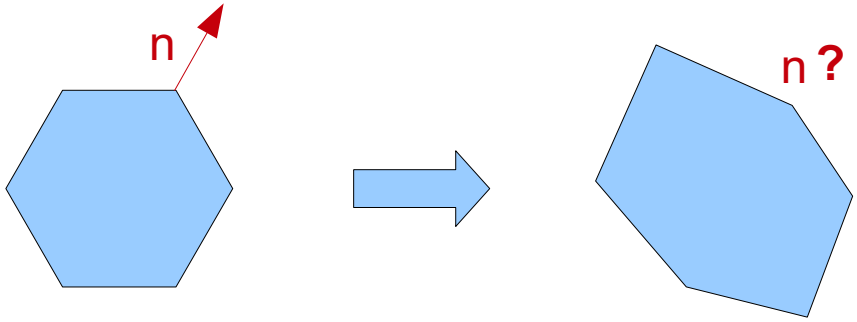
1 - Object Space

4 - Clip Space (D3D)

Vertex shader : Normales

Un vertex traité par la fonction de vertex shader ne connait pas ses vertices avoisinants !

De ce fait, il est difficile de travailler sur la position des vertices en gardant des normales de vertex cohérentes.

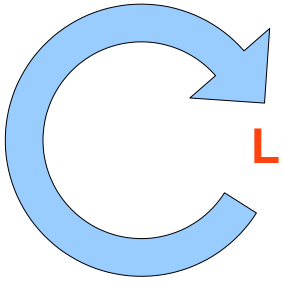


Geometry shader

La fonction de **geometry shader** traite **chaque triangle** du mesh **indépendamment.**

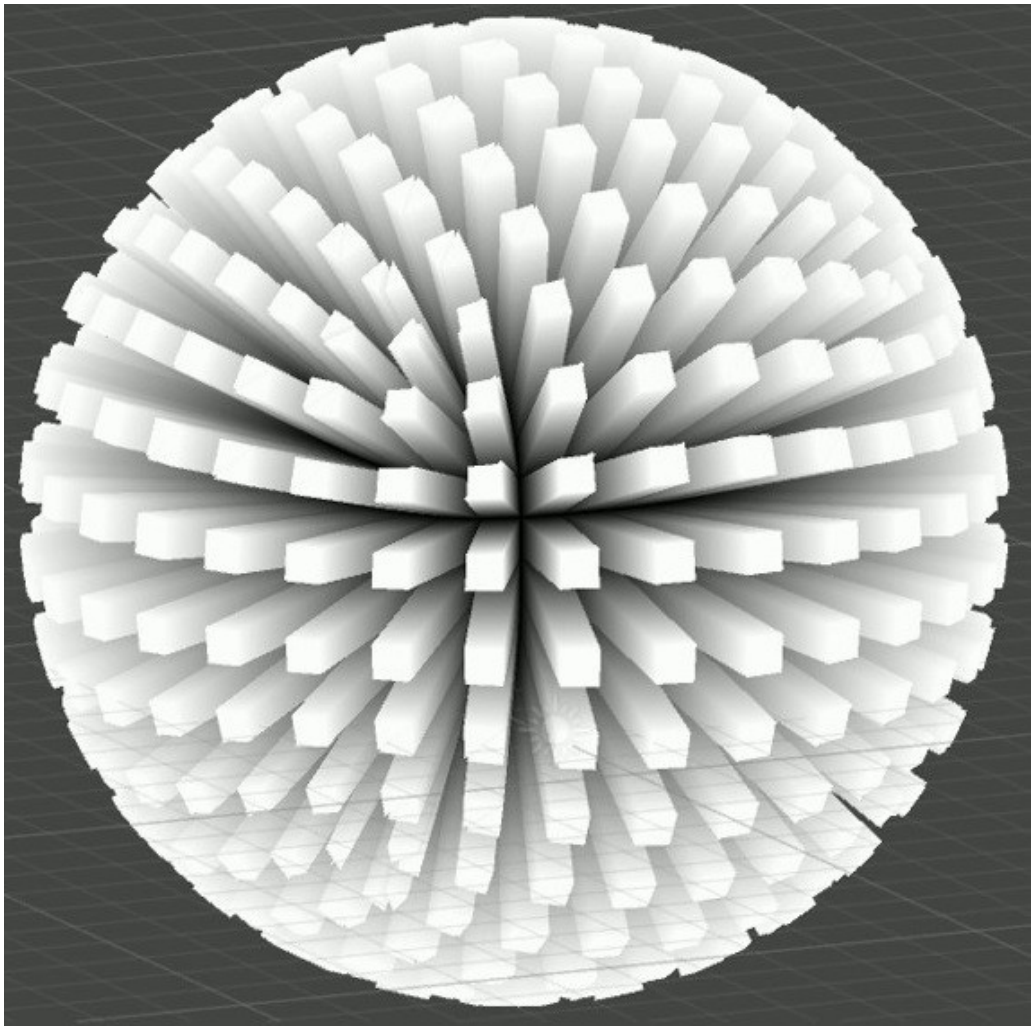
Elle reçoit en entrée un triangle (trois vertices) et renvoie en sortie une liste de triangles. Cette fonction peut donc créer des triangles.

ForEach (TriangleData t in triangles)



List<OutTriangle> o = geom(t)

Geometry shader

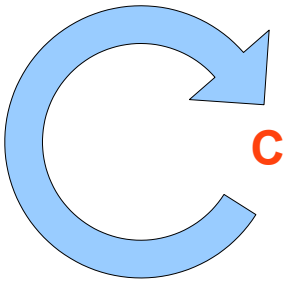


Fragment shader

La fonction de **fragment shader** (ou pixel shader) traite **chaque pixel** occupé par le mesh sur la surface de rendu **indépendamment.**

Elle reçoit les données de vertex interpolées pour un pixel donné et renvoie la couleur à écrire sur ce pixel.

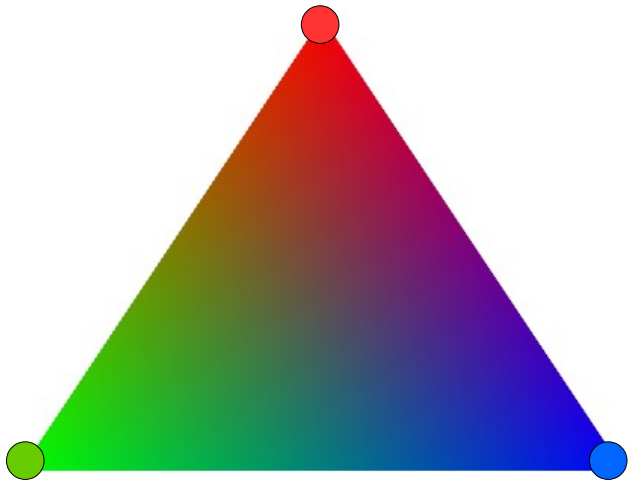
ForEach (PixelData p in drawnPixels)



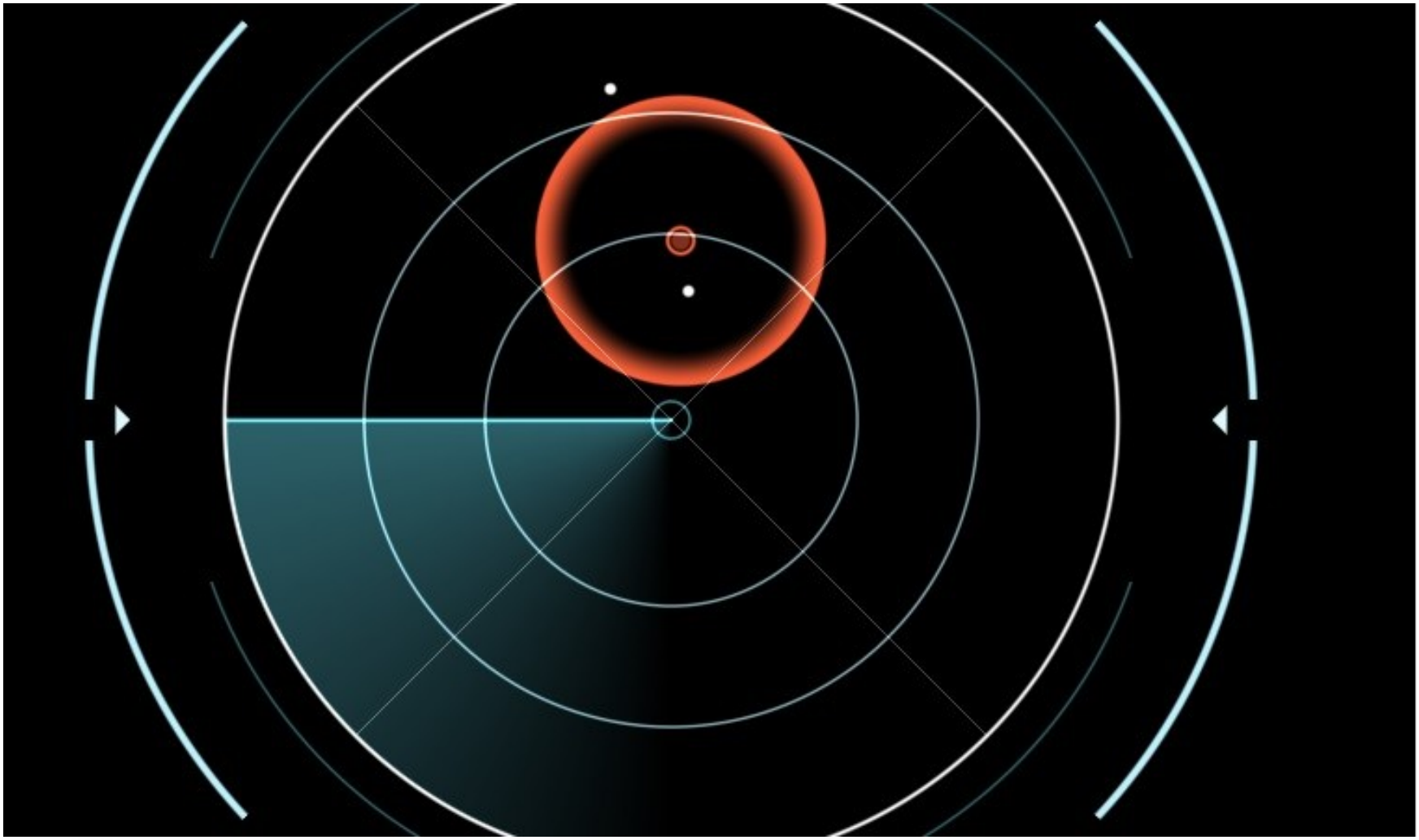
Color o = frag(p)

Rappel : Interpolation

Lors du rendu, les données de points composant un triangle sont interpolées linéairement sur la surface du triangle.



Fragment shader



Fragment shader

Le **fragment shader** traite uniquement les pixels occupés par l'objet 3D sur la surface de rendu.

On ne peut donc pas tracer hors de l'espace occupé par l'objet 3D !

ShaderLab

ShaderLab

ShaderLab est le langage de programmation de shader de Unity. Il se base sur le langage Cg (HLSL).

Il propose entre-autre les fonctionnalités suivantes :

- Compilation multi-plateforme et performante,
- Paramètres accessibles en écriture dans l'inspecteur et par les scripts Unity,
- Nombreuses variables et fonctions "Built-in", ...

Syntaxe

Les langages shaders sont très rigides. Voici quelques différences entre les langages Cg et C# :

- **If** est supporté mais **pas de Switch/Case** en Cg.
- **For** et **While** ne sont supportés qu'à partir du moment où le **nombre d'itérations est constant**.
- La taille d'un **tableau est limitée**.
- Pas de chaînes de caractère, d'enums, de bitfield ni de variables non-signées en Cg.

Limitations mémoires

Le nombre d'instructions et de variables est limité en fonction de la version du moteur 3D utilisé.

A la compilation, les comportements suivants sont probablement attendus :

- Une **fonction** sera copiée "**in-line**" à l'endroit du code où elle a été appelée.
- Une **boucle For** sera "**dépliée**" et son code copié autant de fois que le nombre d'itérations.

Compatibilité

La compatibilité d'un shader dépend :

- Du moteur de rendu (openGL, DirectX, Metal, ...),
- Du matériel graphique visé,
- Des propriétés d'affichage (Pipeline, catégorie d'objets, ...).

Certaines opérations et fonctionnalités ne sont donc pas disponibles en fonction de la configuration software/hardware.

Compatibilité

Unity essaye au maximum de résoudre ces problèmes de compatibilité à la compilation du code ShaderLab.

Dans le cas où Unity n'arrive pas à résoudre une incompatibilité, on peut manuellement écrire plusieurs versions du code d'un même shader grâce aux **SubShaders**.

Propriétés de rendu

Le shader est écrit pour un rendu spécifique et il convient d'avoir un shader adapté !

Par exemple :

- Eclairage : Lit ou Unlit
- Prise en compte de la profondeur
- Transparent ou Opaque
- Sprite, PBR, système de particule,
- Pipeline deferred ou forward, ...

Compilation temps réel

ShaderLab est un langage qui supporte la compilation temps réel (Just In Time).

Un shader peut donc être modifié sans lancer le jeu ou pendant l'exécution du jeu.

Certains effets visuels nécessitent que le jeu soit lancé pour être visibles.

Shader simple

Simple Unlit Unlit

```
Shader "CustomShader/MyShader"
{
    Properties
    {
        _MainTex ("Texture", 2D) = "white" {}
    }
    SubShader
    {
        Tags { "RenderType"="Opaque" }
        LOD 100

        Pass
        {
            CGPROGRAM
            #pragma vertex vert
            #pragma fragment frag
            #pragma multi_compile_fog
            #include "UnityCG.cginc"

            struct appdata
            {
                float4 vertex : POSITION;
                float2 uv : TEXCOORD0;
            };

            struct v2f
            {
                float2 uv : TEXCOORD0;
                UNITY_FOG_COORDS(1)
                float4 vertex : SV_POSITION;
            };

            sampler2D _MainTex;
            float4 _MainTex_ST;

            v2f vert (appdata v)
            {
                v2f o;
                o.vertex = UnityObjectToClipPos(v.vertex);
                o.uv = TRANSFORM_TEX(v.uv, _MainTex);
                UNITY_TRANSFER_FOG(o,o.vertex);
                return o;
            }

            fixed4 frag (v2f i) : SV_Target
            {
                fixed4 col = tex2D(_MainTex, i.uv);
                UNITY_APPLY_FOG(i.fogCoord, col);
                return col;
            }
            ENDCG
        }
    }
}
```

Header

```
Shader "CustomShaders/MyShader"
{
    ...
}
```

Déclare un nouveau shader présent dans la catégorie "CustomShaders" et nommé "MyShader" dans la liste des shaders Unity.

Properties

```
Shader "CustomShaders/MyShader"
{
    Properties
    {
        _MainTex ("Texture", 2D) = "white" {}
    }
```

La variable "_MainTex", plus tard déclarée dans le shader, sera exposée en écriture sous le nom de "Texture" dans l'inspecteur de Unity.

C'est une texture 2D par défaut blanche.

Une Property n'est pas une déclaration de variable ! La variable "_MainTex" sera déclarée plus tard dans le code.

Properties

Les **properties** exposent en écriture des variables du shader dans l'éditeur de Unity. Les types suivants peuvent être exposés :

- Int, Float et Range (float entre deux bornes)
- Colors et Vectors (4 valeurs)
- Textures : 2D, 3D et Cube

Une valeur par défaut doit être attribuée à une property.

SubShader

```
...

SubShader
{
    Tags { "Queue"="Geometry" }
    LOD 100
    ...
}
```

On déclare un premier SubShader.

Il sera ordonné dans la RenderQueue avec la priorité "Geometry" (objets opaques).

LOD permet de filtrer les sub-shaders en fonction d'un niveau de détail demandé en script pour rendre un objet donné.

SubShader

Un sub-shader est une variation du shader qui sera utilisé si les propriétés d'affichage correspondent à ses conditions d'exécution.

On peut par exemple avoir un sub-shader "haute qualité" et un "basse qualité" en fonction du matériel ciblé.

Tags

Les tags permettent de configurer le subshader :

- RenderPipeline : Pipeline cible
- Queue : Position du DrawCall dans la Render Queue
- ForceNoShadowCasting
- DisableBatching
- ...

Pass

```
...

SubShader
{
    Tags { "Queue"="Geometry" }
    LOD 100
    Pass
    {
        CGPROGRAM
        ...
        ENDCG
    }
}
```

Une première passe du shader est déclarée.

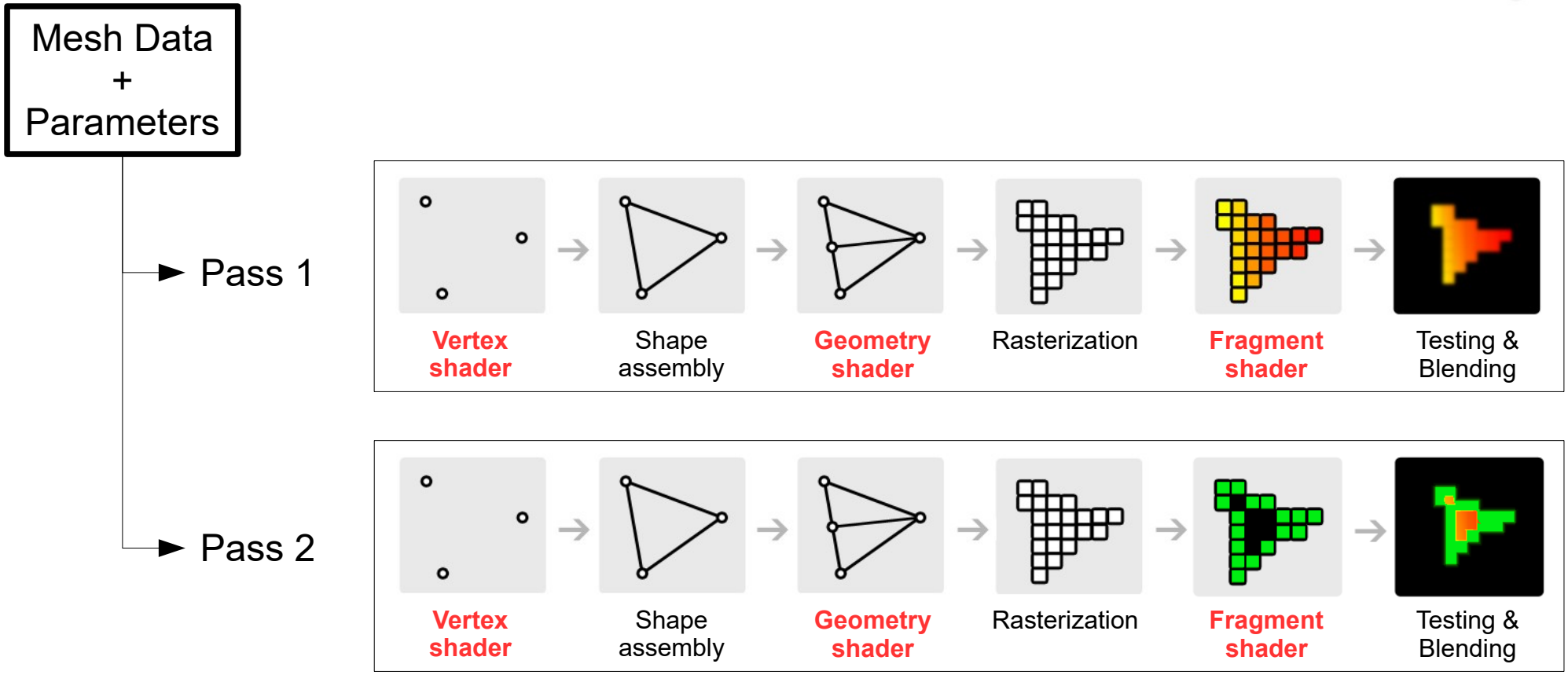
Pass

Un shader peut définir une ou plusieurs passes.

Pour chaque passe, l'ensemble des étapes d'exécution d'un shader sont réalisées.

La seconde passe écrit son rendu sur le résultat de la première.

Pass


Mesh Data
+
Parameters
Pass 1
Vertex shader
Shape assembly
Geometry shader
Rasterization
Fragment shader
Testing & Blending
Pass 2
Vertex shader
Shape assembly
Geometry shader
Rasterization
Fragment shader
Testing & Blending

CGPROGRAM / ENDCG

Le code entre CGPROGRAM et ENDCG est en langage CG (HLSL).

Pass configuration

```
...

Pass
{
    CGPROGRAM
    Cull Off
    ZWrite On
    Blend Off
    ...
    ENDCG
}
...
```

La passe est configurée. Le backface culling est désactivé, la profondeur prise en compte, ...

Pass Commands

En début de passe, on configure (Commands) :

- L'utilisation du "face culling" (On/Off)
- L'écriture dans le z-buffer (On/Off)
- La formule de blend (additif, multiplicatif, ...)
- ...

Une valeur par défaut existe pour chaque option.

Vertex et fragment

```
...

Pass
{
    ...
    #pragma vertex vert
    #pragma fragment frag
    ...
}
```

Les fonctions nommées "vert" et "frag" seront utilisées respectivement pour exécuter le vertex et le fragment shader.

Include UnityCG

```
...

Pass
{
    ...
    #include "UnityCG.cginc"
    ...
}
```

Les fonctions et variables de base de UnityCG sont ajoutées au shader.

Structures de données

```
...

Pass
{
    ...
    struct appdata
    {
        float4 vertex : POSITION;
        float2 uv : TEXCOORD0;
    };
    ...
}
```

La structure provenant du mesh contiendra une position et une coordonnée uv.

Structures de données

```
...

Pass
{
    ...
    struct v2f
    {
        float2 uv : TEXCOORD0;
        UNITY_FOG_COORDS(1)
        float4 vertex : SV_POSITION;
    };
    ...
}
```

La structure passée du vertex shader au fragment shader contiendra une position, une coordonnée uv et une information pour le fog.

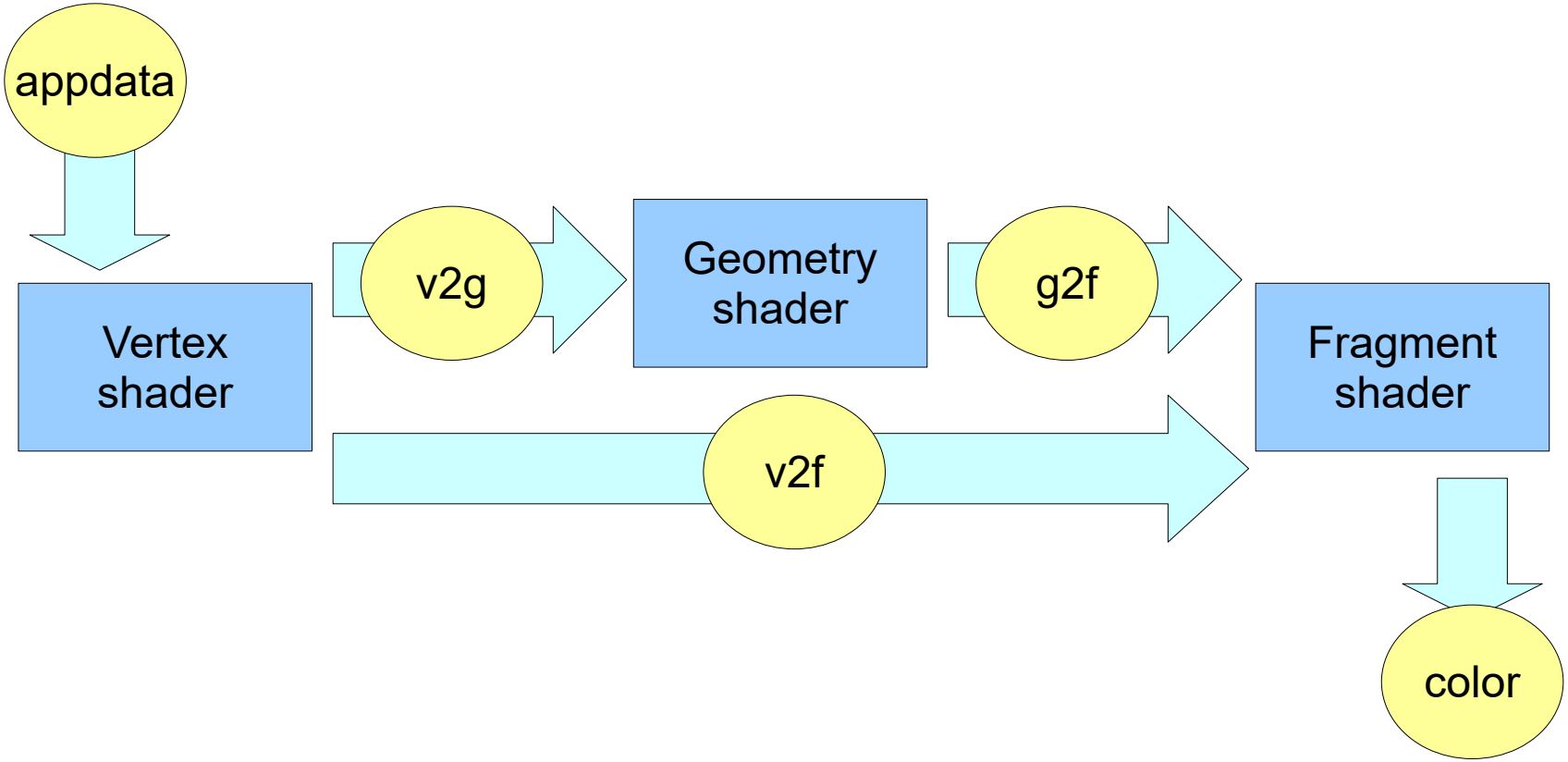
Structures de données

Le shader utilisera plusieurs structures de données pour transférer les données d'une fonction à l'autre.

Elles sont généralement nommées :

- **appdata** : données de vertex du mesh envoyées au vertex shader (aussi nommé "**Attributes"**).
- **v2g** : Données transférées du vertex au geometry shader,
- **g2f** : Données transférées du geometry au fragment shader,
- **v2f** : Données transférées du vertex au fragment shader (pas de geometry shader) (aussi nommé "**Varying"**).

Structure de données



Structures de données

Les structures de données attendent des variables spécifiques avec des **sémantiques** prédéfinies, toutes optionnelles. Pour "appdata" :

- POSITION (float4)
- NORMAL (float3)
- TEXCOORD0, TEXCOORD1 (float4)
- TANGENT (float4)
- COLOR (float4)

Structures de données

Pour "v2f", les **sémantiques** sont :

- SV_POSITION (float4)
- NORMAL (float3)
- TEXCOORD0, TEXCOORD1 (float4)
- TANGENT(float4)
- COLOR0, COLOR1(float4)
- Autres données custom

Variables

```
...

Pass
{
    ...
    sampler2D _MainTex;
    ...
}
```

Déclaration d'une variable globale stockant une texture.

Cette variable est exposée par la propriété déclarée plus tôt.

Variables

Les variables, globales ou locales, peuvent être de types (non-exhaustif) :

- float, half, (fixed) : Nombres décimales par ordre de précision (32, 16 et 11 bits)
- int, char, bool
- float2, float3, int4, ... : vecteurs de valeurs d'un type donné (limité à 4).
- Sample2D : Texture

Variables : type4

Les vecteurs de type float2, bool3, int4, ... ont des accesseurs simplifiés :

- .x, .y, .z et .w pour accéder aux valeurs en index 0, 1, 2 et 3.
- .r, .g, .b et .a synonymes de .x, .y, .z, .w
- .xy, .xzw, .xxx, .yzzw, ... pour accéder à plusieurs valeurs, différentes ou identiques (limité à 4 lettres).

Pour initaliser une variable de type float3, on écrit :

```
float3 name = float3(0.5, 1, -3.7);
```

Fonction de vertex

```
...

Pass
{
    ...
    v2f vert (appdata v)
    {
        v2f o;
        o.vertex = UnityObjectToClipPos(v.vertex);
        o.uv = TRANSFORM_TEX(v.uv, _MainTex);
        UNITY_TRANSFER_FOG(o,o.vertex);
        return o;
    }
    ...
}
```

Transforme Appdata en v2f pour chaque vertex.

Fonction de fragment

```
...

Pass
{
    ...
    fixed4 frag (v2f i) : SV_Target
    {
        fixed4 col = tex2D(_MainTex, i.uv);
        UNITY_APPLY_FOG(i.fogCoord, col);
        return col;
    }
    ...
}
```

Transforme v2f en couleur pour chaque pixel.

Fonction de fragment

La fonction tex2D récupère la couleur d'un pixel sur une texture en fonction de coordonnées UV.

La couleur renvoyée par la fonction de fragment sera combinée à la couleur déjà existante de ce pixel sur le frame buffer en utilisant le blend défini par la passe.

Fin

```
Shader "CustomShaders/MyShader"
{
    ...
    SubShader
    {
        ...
        Pass
        {
            ...
            ENDCG
        }
        ...
    }
}
```

Si une autre passe existe dans ce SubShader elle est exécutée. Sinon, le shader se termine.

Fonctions et Variables built-in

Fonctions : Math

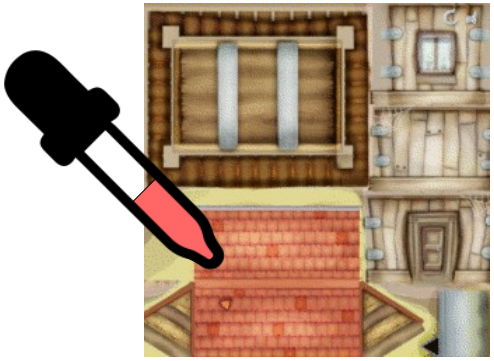
- **sin, cos, tan, abs, atan2, dot, cross, distance, min, max, sign, pow, sqrt, lerp, ...**

  Un grand nombre de fonctions mathématiques sont présentes nativement en Cg.

Fonctions : Sampling

- **float4 tex2D(sampler2D, float2)**

  Dans un fragment shader, récupère une couleur sur une texture pour une coordonnée uv.



Fonctions

- **discard;**

  Ignore le pixel, arrête immédiatement la fonction de vertex shader pour ce pixel.

- **clip(float)**

  Ignore le pixel si la valeur en paramètre est inférieure à 0. Arrête la fonction de vertex.

Variables built-in

- **_Time.y :** Le temps depuis le chargement de la scène.
- **_ScreenParams :** Les informations sur la taille du frame buffer.
  - **_ScreenParams.xy :** Largeur et hauteur de l'écran
  - **_ScreenParams.zw : (**1 / largeur) et (1 / hauteur)
- **UNITY_MATRIX_[...] :** Les matrices de transformation utilisées par le shader.

Grab Pass

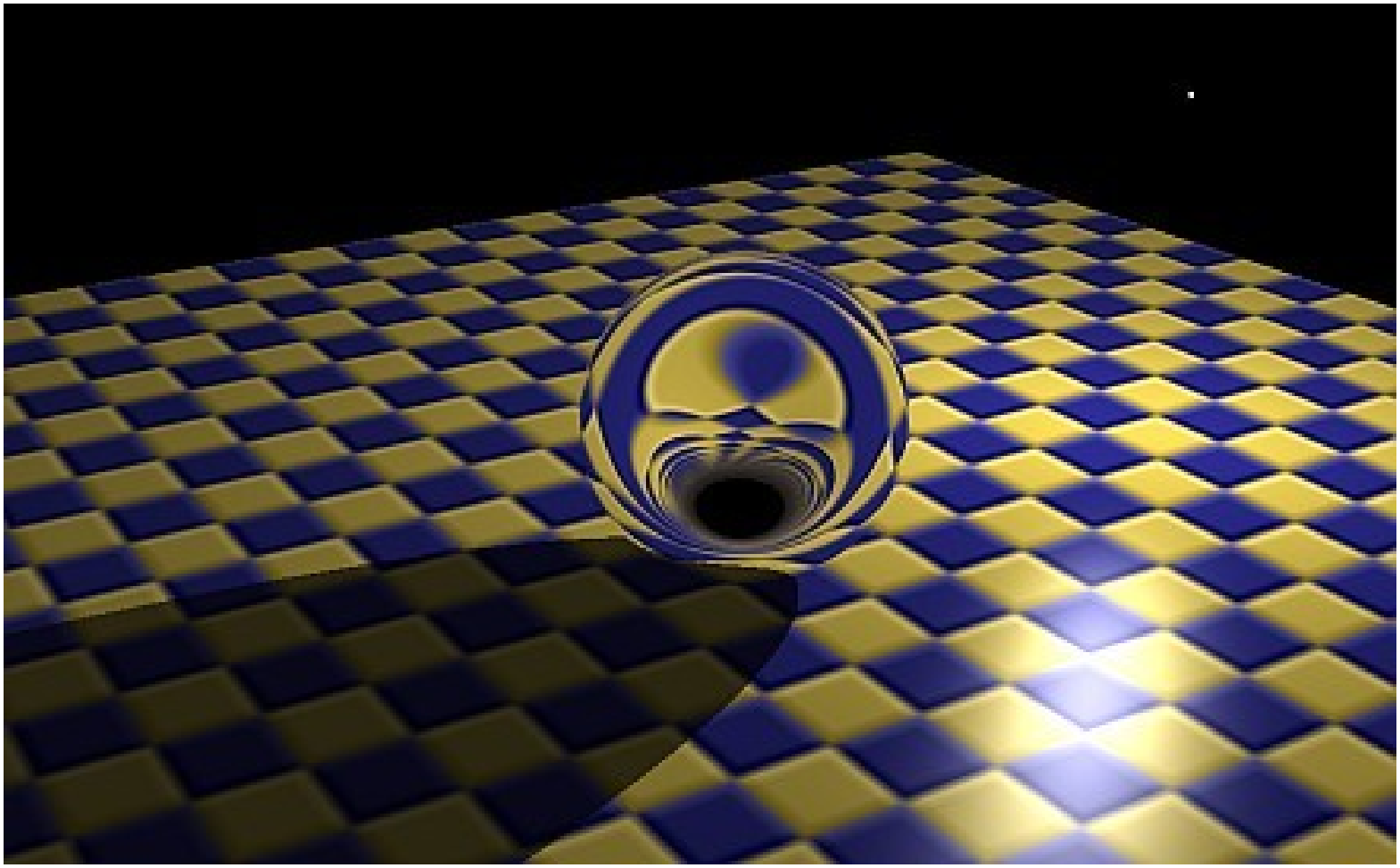
Grab Pass

La Grab Pass est une passe de shader pré-existante.

Elle permet de récupérer dans une texture les valeurs de couleur du frame buffer.

Elle est essentielle pour réaliser certains effets de distorsion type réfraction.

Grab Pass



Grab Pass

Avant la première passe de notre shader, on écrit la passe suivante :

```
GrabPass { } // Keep {} empty.
```

La variable global de texture **_GrabTexture** est alors automatiquement déclarée et remplie.

Grab Pass

float4 ComputeGrabScreenPos(float4)

La fonction ci-dessus prend en paramètre une position en **projection Clip** et la convertie en **projection Screen.**

Il faut **diviser cette position par les dimensions** du frame buffer pour obtenir les coordonnées d'UV sur la _GrabTexture.

Stencil buffer

Stencil Buffer

Le **stencil buffer** est une donnée de pixel sur le frame buffer écrite et de partagée par les shaders.

Il est **unique et commun pour tous les Draw Calls.**

Comme le depth buffer, il permet d'afficher ou d'ignorer des pixels à rendre.

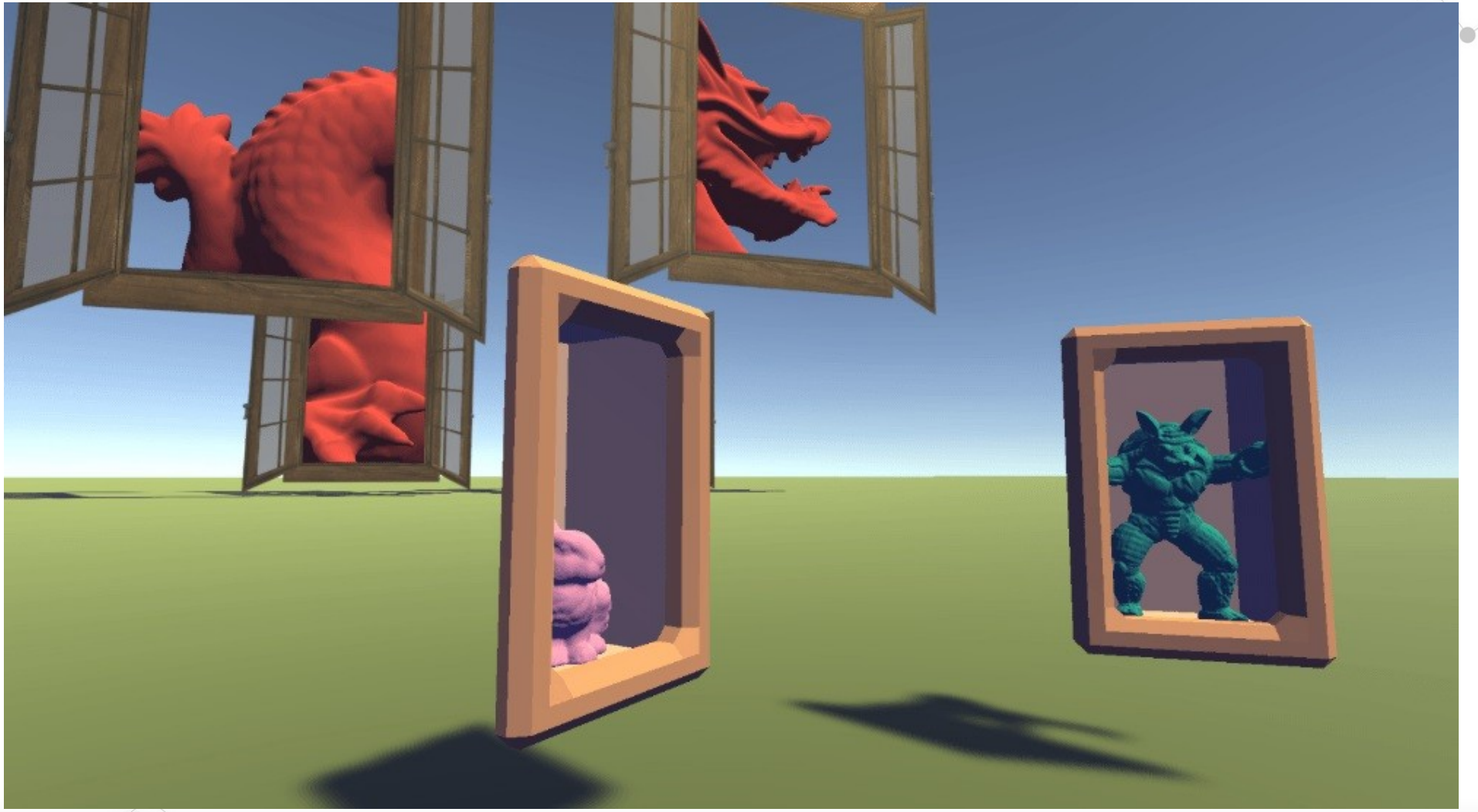
Stencil Buffer

Le stencil buffer est une valeur sur 8 bits entre 0 et 255. Par défaut, il vaut 0.

Le stencil buffer n'est lu, comparé et écrit qu'avant l'appel au fragment shader.

On ne peut pas manipuler le stencil buffer dans les fonctions.

Stencil Buffer



Stencil Buffer

Les règles d'utilisation du stencil buffer sont déclarées par passe.

```
...
Pass {
    Stencil {
        Ref 2
        Comp always
        Pass replace
    }
    ...
}
```

Stencil Buffer

```
Pass {
    Stencil {
        Ref 2 // Comparison value : 2
        Comp always // Comparison condition : always perform pass
        Pass replace // Action : If passed, replace previous value by 2
    }
    ...
}

Pass {
    Stencil {
        Ref 1 // Comparison value : 1
        Comp greater // Comparison condition : perform pass if greater
        Fail incrSat // Action : If failed, increment previous value
    }
    ...

}
```

Surface Shader

Surface Shader

Les shaders utilisant les lumières peuvent être très complexes : Types de lumières, ombres, rendering path, ...

De manière à simplifier l'écriture de shaders pour objets 3D éclairés, Unity propose une syntaxe de simplifiée* : Le surface shader.

*Note that there are no custom languages, magic or ninjas involved in Surface Shaders – Documentation Unity 2019.1

Surface Shader

Le Surface Shader permet de se concentrer sur une fonction de surface shader.

Cette fonction traite et converti les données de v2f interpolées pour chaque pixel avant éclairage.

Cette fonction sera intégrée à la compilation dans un fragment shader complexe.

Surface Shader

Surface Shader

Le Surface Shader nous laisse la possibilité de choisir un modèle d'éclairage : Standard, Lambert, BlinnPhong, ...

Il nous permet également manipuler facilement les ombres, la tessellation, le blend, ... avec des modèles prédéfinis.

Ces modèles seront intégrés à la compilation.

Surface Shader

Le surface shader nous permet d'intercaler des fonctions custom :

- Au début du vertex shader, pour modifier les données en entrée.
- A la fin du fragment shader, pour modifier la couleur en sortie.

Surface Shader

```
Shader "Example/Diffuse Simple" {
	SubShader {
		Tags { "RenderType" = "Opaque" }
		CGPROGRAM
		#pragma surface surf Lambert

		struct Input {
			float4 color : COLOR;
		};
		void surf (Input IN, inout SurfaceOutput o) {
			o.Albedo = 1;
		}
		ENDCG
	}
	Fallback "Diffuse"
}
```

SRP...

SRP

Il est actuellement (Unity 2022) très difficile d'écrire un shader en intégrant correctement la lumière et les effets standards dans SRP.

- ShaderLab n'est pas compatible SRP.
- La meilleure option reste souvent Shader Graph.

SRP

ShaderGraph ne supporte pas encore certaines features telles que la Tesselation et le Stencil Buffer.

Bibliographie pour écrire un shader SRP en script :

- https://www.cyanilux.com/tutorials/urp-shader-code/
- https://nedmakesgames.medium.com/writing-unity-urp-shaders-with-code-part-1-the-graphics-pipeline-and-you-798cbc941cea